

# The Grand Cheat Sheet

## In This Edition

---

|    |                                                             |   |
|----|-------------------------------------------------------------|---|
| 1  | 1. Concurrency & Multithreading                             | 2 |
| 2  | 2. Operating Systems                                        | 2 |
| 3  | 3. Computer Networks                                        | 2 |
| 4  | 4. Cloud & Infrastructure                                   | 3 |
| 5  | 5. System Design                                            | 3 |
| 6  | 6. Low-Level Design (LLD)                                   | 4 |
| 7  | 7. Distributed Systems                                      | 4 |
| 8  | 8. Databases                                                | 4 |
| 9  | 9. Performance Engineering                                  | 5 |
| 10 | 10. Security Fundamentals                                   | 5 |
| 11 | 11. Behavioral / Leadership / Googleyness                   | 6 |
| 12 | Cross-Topic Memory Hooks                                    | 6 |
| 13 | The 12 Highest-Frequency Interview Concepts (memorize cold) | 6 |

Dense, high-signal recall. For depth, open the per-topic file. Read top-to-bottom in ~20 min.

## 1. Concurrency & Multithreading

- › **Concurrency**  $\neq$  **Parallelism**: concurrency = *dealing with* many things (structure); parallelism = *doing* many things at once (execution).
- › **Process** = own memory space; **Thread** = shared memory, own stack. Threads cheaper, but share state  $\rightarrow$  bugs.
- › **Race condition**: outcome depends on timing of unsynchronized access to shared state. Fix with synchronization.
- › **Mutex** = lock, 1 owner (mutual exclusion). **Semaphore** = counter, N permits. **Binary semaphore**  $\approx$  **mutex** (but no ownership). **Condition variable** = wait/signal on a predicate.
- › **Deadlock 4 conditions (must ALL hold)**  $\rightarrow$  mnemonic "MHNC": **M**utual exclusion, **H**old & wait, **N**o preemption, **C**ircular wait. Break any one to prevent. (Order locks globally  $\rightarrow$  kills circular wait.)
- › **Livelock** = threads act but no progress; **Starvation** = thread never scheduled.
- › **Producer-Consumer**: bounded buffer + 2 semaphores (empty, full) + mutex. **Reader-Writer**: many readers OR one writer.
- › **CAS / lock-free**: atomic compare-and-swap, no locks; **ABA problem** lurks. **volatile** = visibility (not atomicity).
- › **Thread pool**: reuse N worker threads from a queue  $\rightarrow$  avoid thread-creation cost; bound concurrency.
- › **Takeaway**: "Shared mutable state + no synchronization = bug." Prefer immutability / message-passing.

## 2. Operating Systems

- › **Process vs Thread**: process = isolation + own address space; thread = shared address space, lighter context switch.
- › **Scheduling**: FCFS (convoy effect), SJF (optimal avg wait, needs prediction), **Round Robin** (fair, time-slice), Priority (starvation $\rightarrow$ aging), **MLFQ** (adaptive), Linux **CFS** (fair via vruntime).
- › **Virtual memory**: every process sees its own contiguous address space; MMU maps virtual $\rightarrow$ physical via **page tables** (+TLB cache).
- › **Paging**: fixed-size pages; **page fault**  $\rightarrow$  load from disk. **Replacement**: Optimal > LRU > Clock > FIFO (Belady's anomaly = FIFO can worsen with more frames).
- › **Thrashing**: too many page faults, CPU stalls on I/O  $\rightarrow$  fix with working-set model / reduce multiprogramming.
- › **Context switch**: save/restore registers + PC + memory maps; expensive (flush TLB). Threads cheaper than processes.
- › **IPC**: pipes, shared memory (fastest), message queues, sockets. **fork()** = copy process (**copy-on-write**); **exec()** = replace image.
- › **File system**: **inode** = file metadata + block pointers; **journaling** = crash consistency.
- › **Takeaway**: kernel/user mode + system calls = the privilege boundary; virtual memory = the great illusion.

## 3. Computer Networks

- › **OSI** (mnemonic "Please Do Not Throw Sausage Pizza Away"): Physical, Data-link, Network, Transport, Session, Presentation, Application.
- › **TCP vs UDP**: TCP = reliable, ordered, connection (3-way handshake **SYN** $\rightarrow$ **SYN-ACK** $\rightarrow$ **ACK**), flow+congestion control. UDP = fast, fire-and-forget (video, DNS, games).

- › **TLS handshake:** ClientHello → ServerHello+cert → key exchange → finished. **Asymmetric to exchange a symmetric session key**, then symmetric for speed. TLS 1.3 = 1-RTT.
- › **DNS resolution:** browser cache → OS → resolver → root → TLD (.com) → authoritative → IP.
- › **REST** (stateless, resources, HTTP verbs, cacheable) vs **gRPC** (HTTP/2, protobuf, streaming, fast internal RPC) vs **GraphQL** (client picks fields).
- › **HTTP/1.1** (head-of-line blocking) → **HTTP/2** (multiplexing, server push, binary) → **HTTP/3** (QUIC over UDP, no TCP HOL blocking).
- › **WebSocket** = full-duplex persistent; **SSE** = server→client stream; **long-polling** = fallback.
- › **L4 LB** (transport, IP/port, fast) vs **L7 LB** (application, content-aware routing).
- › **Idempotency:** GET/PUT/DELETE idempotent; POST not.
- › **"Type google.com → ?":** DNS → TCP handshake → TLS → HTTP GET → server → render. Know every hop.

## 4. Cloud & Infrastructure

- › **Container vs VM:** container shares host kernel (namespaces=isolation, cgroups=limits), MBs, secs; VM = full OS, GBs, mins.
- › **Docker image** = layered (union FS), immutable; container = running instance.
- › **Kubernetes:** control plane (**API server, etcd, scheduler, controller-mgr**) + nodes (**kubelet, kube-proxy**). Pod = smallest unit; Deployment = replicas+rollout; Service = stable virtual IP; Ingress = L7 routing.
- › **Autoscaling:** HPA (pods by metric), VPA (resize), Cluster Autoscaler (nodes).
- › **Service mesh** (Istio/Envoy sidecar): mTLS, retries, traffic-splitting, observability — off the app code.
- › **CI/CD deploys:** **Rolling** (gradual), **Blue-Green** (instant switch, easy rollback), **Canary** (small % first).
- › **IaC:** Terraform (declarative, multi-cloud) vs CloudFormation (AWS).
- › **Observability = 3 pillars:** **Metrics** (Prometheus, aggregate), **Logs** (events), **Traces** (request across services). **SLI** (measured) → **SLO** (target) → **SLA** (contract+penalty); **error budget** = 1–SLO.
- › K8s = "desired state" reconciliation loop. You declare; controllers converge.

## 5. System Design

- › **Framework:** Requirements (functional + non-functional) → Estimation → API → Data model → High-level diagram → Deep dive → Bottlenecks/trade-offs.
- › **Scale:** **Vertical** (bigger box, limit + SPOF) vs **Horizontal** (more boxes, needs statelessness + LB). **Stateless** scales easily.
- › **Caching strategies:** **Cache-aside** (lazy, app manages), **Write-through** (write cache+db together), **Write-back** (write cache, async db — fast but risky), **Write-around**. Eviction: **LRU/LFU**. Hardest problem = **invalidation**.
- › **CDN:** cache static content at edge, near users.
- › **Sharding** (partition data horizontally; watch hot partitions) + **Replication** (copies; leader-follower = read scaling, multi-leader = write availability).
- › **CAP:** on partition, pick **C** (reject) or **A** (serve stale). CA only without partitions (single node). **PACELC:** else, latency vs consistency.
- › **Message queues** (Kafka/SQS/RabbitMQ): decouple, buffer, smooth spikes, async. **Event-driven** = react to events, loose coupling.
- › **Microservices:** independent deploy/scale; cost = distributed complexity, network, data consistency.
- › **Latency numbers:** L1≈1ns, RAM≈100ns, SSD≈100µs, disk seek≈10ms, network RT (same DC)≈0.5ms, cross-continent≈150ms.
- › Always state assumptions + estimates + trade-offs out loud. Silence loses points.

## 6. Low-Level Design (LLD)

- › **OOP 4 pillars** (mnemonic "A PIE"): Abstraction, Polymorphism, Inheritance, Encapsulation.
- › **SOLID**:
  - Single Responsibility — one reason to change.
  - Open/Closed — open to extend, closed to modify.
  - Liskov Substitution — subtype usable as base type.
  - Interface Segregation — small focused interfaces.
  - Dependency Inversion — depend on abstractions, not concretions.
- › **Composition > Inheritance** ("has-a" > "is-a"); low coupling, high cohesion; DRY/KISS/YAGNI.
- › **Patterns** (by category):
  - **Creational**: Singleton, Factory, Abstract Factory, Builder, Prototype.
  - **Structural**: Adapter, Decorator, Facade, Proxy, Composite.
  - **Behavioral**: Strategy, Observer, Command, State, Template Method, Iterator.
- › **UML relationships**: inheritance ( $\leftarrow$ ), composition ( $\blacklozenge$ ), owns lifecycle ( $\blacklozenge$ ), aggregation ( $\diamond$ ), association ( $--$ ), dependency ( $- - >$ ).
- › LLD approach: clarify  $\rightarrow$  nouns=classes, verbs=methods  $\rightarrow$  relationships  $\rightarrow$  apply patterns  $\rightarrow$  code core  $\rightarrow$  discuss extensibility. Don't over-pattern.

## 7. Distributed Systems

- › **8 Fallacies**: network is reliable, latency=0, bandwidth  $\infty$ , network secure, topology stable, one admin, transport cost=0, network homogeneous — *all false*.
- › **CAP**  $\rightarrow$  see #5. **Consistency models**: Strong > Linearizable > Causal > Read-your-writes > Eventual.
- › **Consistent hashing**: hash nodes+keys onto a ring; adding/removing a node moves only  $\sim K/N$  keys. **Virtual nodes** smooth load.
- › **Quorum**:  $R + W > N \rightarrow$  strong consistency (overlap guarantees latest read).  $W=N$  (durable),  $R=1$  (fast reads).
- › **Consensus** (agree on one value despite failures): **Raft** = leader election (terms, votes) + log replication + safety (commit when majority). **Paxos** = correct but notoriously hard to grok. Used by etcd/ZooKeeper/Spanner.
- › **Leader election**: avoid **split-brain** with quorum + fencing tokens.
- › **Vector clocks / Lamport timestamps**: order events / detect concurrent updates.
- › **2PC** (blocking, coordinator SPOF) vs **Saga** (local txns + compensations, eventual).
- › **Delivery**: at-most-once, at-least-once (retries  $\rightarrow$  dedup needed), **"exactly-once"** = **at-least-once + idempotency** (true E2E is a myth).
- › **Kafka**: topic  $\rightarrow$  partitions (ordered, parallel)  $\rightarrow$  consumer groups; offset-based, replayable, durable log.
- › Real systems: DynamoDB/Cassandra = AP/tunable; Spanner = CP (TrueTime); etcd/ZK = consensus.

## 8. Databases

- › **ACID**: Atomicity (all-or-nothing), Consistency (valid state), Isolation (concurrent = serial-ish), Durability (survives crash).
- › **Isolation levels**  $\times$  **anomalies**:

| Level            | Dirty Read | Non-Repeatable | Phantom           |
|------------------|------------|----------------|-------------------|
| Read Uncommitted | possible   |                |                   |
| Read Committed   |            |                |                   |
| Repeatable Read  |            |                | (MySQL: via MVCC) |
| Serializable     |            |                |                   |

- › **Indexes:** **B+tree** (range queries, reads, RDBMS) vs **LSM-tree** (write-heavy, Cassandra/RocksDB; memtable→SSTables→compaction). **Clustered** = data sorted by key (1/table); **non-clustered** = separate pointer structure.
- › **Joins:** INNER (both), LEFT (all left + matches), RIGHT, FULL, CROSS (cartesian).
- › **CTE** = WITH x AS (...) readable/recursive. **Window functions** = OVER(PARTITION BY ... ORDER BY ...) (ROW\_NUMBER, RANK, LAG/LEAD) — aggregate *without* collapsing rows.
- › **Locking:** **Optimistic** (version check at commit, low contention) vs **Pessimistic** (lock upfront, high contention). **MVCC** = readers don't block writers.
- › **SQL vs NoSQL:** SQL = relations, ACID, joins, vertical. NoSQL families: **KV** (Redis), **Document** (Mongo), **Column-family** (Cassandra), **Graph** (Neo4j) — denormalize, scale horizontally, BASE.
- › Index = faster reads, slower writes + storage. Don't index everything.

## 9. Performance Engineering

- › **Latency** (time/request) vs **Throughput** (requests/sec) vs **Bandwidth** (max capacity). Optimize for the one that matters.
- › **Tail latency:** track **p50/p90/p99/p999** — averages lie. At scale, p99 hits most users on a fan-out request (Google "Tail at Scale").
- › **Little's Law:**  $L = \lambda \times W$  (items-in-system = arrival-rate  $\times$  time-in-system). **Amdahl's Law:** speedup capped by serial fraction.
- › **USE** (resources: Utilization, Saturation, Errors) + **RED** (services: Rate, Errors, Duration).
- › **Bound types:** CPU-bound (high util, profile hot path), I/O-bound (waiting, batch/async), memory-bound (cache misses, GC), network-bound.
- › **Memory hierarchy** (cache locality matters): register→L1(1ns)→L2→L3→RAM(100ns)→SSD(100μs)→disk(10ms)→
- › **Workflow:** **Measure** → **Profile (flame graph)** → **find bottleneck** → **fix ONE thing** → **re-measure**. "Premature optimization is the root of all evil."
- › **Memory leak:** heap grows unbounded; find via heap dumps / allocation profiling.
- › **Speed levers:** caching, batching, connection pooling, pipelining, async, reducing N+1.
- › "Service got 10× slower" → check recent deploys, metrics (USE/RED), profile, narrow CPU vs I/O vs lock contention vs downstream.

## 10. Security Fundamentals

- › **AuthN** = *who are you* (identity) vs **AuthZ** = *what can you do* (permissions). AuthN before AuthZ.
- › **CIA triad:** Confidentiality, Integrity, Availability.
- › **Encryption vs Hashing vs Encoding:** encryption = reversible w/ key (confidentiality); hashing = one-way (integrity, passwords); encoding = not security (Base64).
- › **Symmetric** (AES, one shared key, fast, bulk data) vs **Asymmetric** (RSA/ECC, public/private, key exchange + signatures, slow).
- › **Password storage:** hash + **unique salt** with **bcrypt/argon2/scrypt** (slow on purpose). Never plaintext/MD5.

- › **OAuth 2.0** = delegated *authorization* (give app access without password); use **Authorization Code + PKCE** for apps. **OIDC** adds *authentication* (ID token) on top. **JWT** = header.payload.signature — signed (not encrypted by default!); verify signature, check exp, don't store secrets in payload.
- › **OWASP Top 10** (know a few cold): Broken Access Control (#1), Cryptographic Failures, **Injection** (SQLi → parameterized queries), Insecure Design, Misconfiguration, Vulnerable Components, Auth Failures, Data Integrity, Logging Failures, SSRF.
- › **Web attacks**: **XSS** (inject script → escape output/CSP), **CSRF** (forged request → anti-CSRF token/Same-Site), **SQLi** (→ prepared statements).
- › **Principles**: least privilege, defense in depth, zero trust, secrets in **Vault/KMS** (never in code/env-in-git), rotate secrets.
- › "Design secure login": TLS + hashed/salted passwords + rate limiting + MFA + secure session/JWT + least privilege.

## 11 Behavioral / Leadership / Googleness

- › **STAR**: Situation (context, brief) → Task (your responsibility) → Action (what **YOU** did, 60%) → Result (quantified impact).
- › Use **"I" not "we"** for your actions. Always end with metrics ("reduced latency 40%", "saved \$200k/yr").
- › **Themes & what they probe**: Leadership (drive without authority), Ownership (end-to-end, beyond your scope), Conflict (disagree respectfully, data-driven, **disagree & commit**), Collaboration (cross-team influence), Ambiguity (act with incomplete info), Mentoring (grow others), Impact (measurable outcomes).
- › **Amazon LPs** (bar raiser): Customer Obsession, Ownership, Invent & Simplify, Dive Deep, Have Backbone; Disagree & Commit, Deliver Results, etc. Map a story to each.
- › **Google "Googleness"**: comfort with ambiguity, collaboration, intellectual humility, doing the right thing.
- › **Story bank**: prep **6–8 stories** that flex across themes (1 conflict, 1 failure, 1 leadership/impact, 1 ambiguity, 1 mentoring, 1 cross-team). Build a story × theme matrix.
- › **Failure question**: pick a *real* failure, own it, show what you *learned/changed*. Never blame others; never a fake-humble "I work too hard."
- › Top mistakes: rambling, no metrics, "we" instead of "I", no failure story, blaming others. Be concise + structured + quantified.

## 12 Cross-Topic Memory Hooks

| Connection                        | Hook                                                                          |
|-----------------------------------|-------------------------------------------------------------------------------|
| OS ↔ Concurrency                  | Threads/processes/scheduling/locks are OS primitives<br>concurrency builds on |
| Networks ↔ System Design          | LB, caching, CDN, protocols = the plumbing of every design                    |
| Databases ↔ Distributed Systems   | CAP, replication, sharding, quorum, consistency overlap heavily               |
| Cloud ↔ Distributed Systems       | K8s/service-mesh = distributed-systems patterns productized                   |
| Performance ↔ everything          | Profiling + tail latency + caching cut across OS, DB, network                 |
| Security ↔ Networks/System Design | TLS, auth, secrets are non-functional requirements in every design            |

## 13 The 12 Highest-Frequency Interview Concepts (memorize cold)

1. CAP theorem + consistency models

2. TCP handshake + TLS + "type a URL" flow
3. Caching strategies + invalidation
4. Sharding + replication + consistent hashing
5. ACID + isolation levels matrix
6. B+tree vs LSM-tree indexes
7. Deadlock 4 conditions ("MHNC")
8. Process vs thread + virtual memory/paging
9. SOLID + composition over inheritance
10. Tail latency (p99) + measure-before-optimize
11. OAuth/JWT + OWASP injection/XSS/CSRF
12. STAR with quantified "I" results

**Golden rule for every round:** state assumptions → think out loud → give the trade-off, not just the answer.